REACT PRIMER — SHEET 01

# Why React?

You just built a task card by hand, straight against the DOM. Before you touch Vite, Tailwind, or a single `npm install` — this is the problem React exists to solve, and why it's shaped the way it is.

SHEET 02 — THE PROBLEM

# This is what you already wrote

DOM_TUTORIAL.MD — CREATETASKCARD()

```
function createTaskCard(title) {
  const article = document.createElement("article");
  article.className = "task-card";

  const heading = document.createElement("h3");
  heading.textContent = title;

  const deleteBtn = document.createElement("button");
  deleteBtn.type = "button";
  deleteBtn.textContent = "Delete";
  deleteBtn.addEventListener("click", () => article.remove());

  article.appendChild(heading);
  article.appendChild(deleteBtn);
  return article;
}
```

- **Every element, by hand.** Three `createElement` calls to describe a card you could write in three lines of HTML.
- **Manual wiring.** Forget one `appendChild` and the card silently loses a piece.
- **Nothing here says what the card should look like** — only the steps to build it. Read it twice to find the shape.

SHEET 03 — NAMED PROBLEMS

# It gets worse than one card

Multiply this by every list, form, and toggle in a real app, and four problems compound:

01

### Imperative steps

You write *how* to build the UI, one instruction at a time — not *what* it should be.

02

### Manual bookkeeping

Every append, every listener, every cleanup is on you to remember and never forget.

03

### No single source of truth

Data lives in variables; the DOM lives in the page. Nothing keeps them in sync automatically.

04

### Doesn't scale

Fine for one card. Unmanageable for a dashboard with a hundred moving pieces.

SHEET 04 — THE IDEA

# Describe the result. Let something else build it.

## UI = f( state )

REACT'S WHOLE PREMISE

You stop writing the steps to mutate the DOM. You write a function that says what the UI looks like *for the current state* — and React figures out which DOM operations make that true.

| | | |
|---|---|---|
| imperative | → | create → mutate → hope it's consistent |
| declarative | → | describe(state) → React reconciles the DOM |

SHEET 05 — JSX

# Same card. Described, not built.

BEFORE — IMPERATIVE DOM

```
document.createElement("article")
  .appendChild(heading)
  .appendChild(deleteBtn)
```

AFTER — JSX COMPONENT

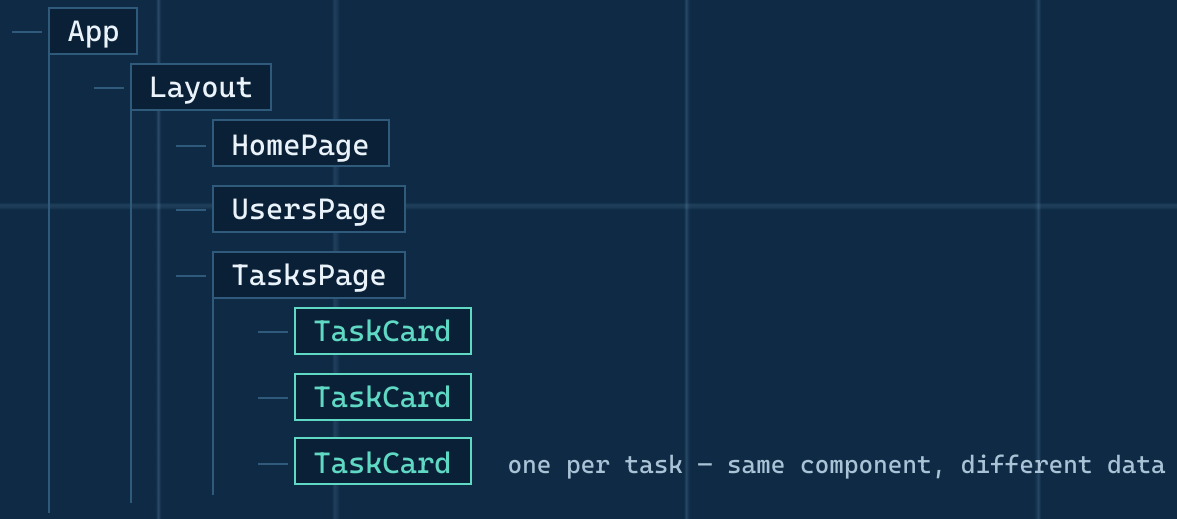
```
function TaskCard({ title, onDelete }) {
  return (
    <article className="task-card">
      <h3>{title}</h3>
      <button onClick={onDelete}>Delete</button>
    </article>
  )
}
```

JSX is HTML-shaped syntax embedded in TypeScript. Both snippets produce the same DOM — one of them reads like the result.

SHEET 06 — COMPONENTS

# Nest small pieces into the whole app

A component is a function that owns one piece of UI. Apps are trees of them — each one reusable and testable on its own.



SHEET 07 — STATE & RE-RENDER

# The loop that replaces manual DOM edits

`user clicks Delete` → `setState()` → `React re-runs the component` → `DOM updates itself`

You never call `.remove()` or `.appendChild()` again. You change the *data*; React changes the *page* to match. The `useState`/`useEffect` hooks you'll meet in the tutorial are how a component reads and reacts to that state.

SHEET 08 — WHERE THIS FITS

# The stack you're about to assemble

| | | |
|---|---|---|
| Vite | Build tool and dev server — the thing running `npm run dev` | Part 1 |
| React | The component model this whole deck has been building toward | Part 1 |
| Tailwind CSS | Utility classes instead of hand-written stylesheets | Part 1 |
| shadcn/ui | Accessible components you own the source of | Part 1 |
| React Router | Swaps components on-screen as the URL changes | Part 1 |
| Axios + TanStack Query | Fetches data and turns it into state components re-render from | Part 2 |

SHEET 09 — START BUILDING

# You now know what problem you're solving.

Every exercise from here on is that same idea in more concrete form: describe the UI, hand state to a component, let React handle the rest.

APPROVED FOR CONSTRUCTION
→ REACT_STYLING_TUTORIAL.MD, PART 1